МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра Название кафедры

ОТЧЕТ

по практической работе № 1
по дисциплине «Системы параллельной обработки сигналов»

Тема: «Запуск параллельной программы на различном числе одновременно работающих процессов. Упорядочение вывода результатов»

Студент гр. 0000 _____ Фамилия И. О.
подпись

Преподаватель _____ Фамилия И. О.
подпись

Санкт-Петербург
2026

1 Номер варианта

Вариант: не указан. В исходном задании отдельный номер варианта не указан.

2 Цель работы

Познакомиться с запуском MPI-программы на различном числе процессов, обменом сообщениями между процессами и способами управления порядком вывода результатов параллельного выполнения.

3 Формулировка задания

Задание 1. Все процессы, кроме процесса с рангом 0, передают значение своего ранга нулевому процессу. Процесс с рангом 0 сначала печатает значение собственного ранга, затем принимает сообщения с рангами остальных процессов и также выводит их. Порядок приёма сообщений заранее не определён и может изменяться от запуска к запуску.

Необходимо несколько раз запустить программу при фиксированном числе процессов (например, при восьми), проанализировать порядок вывода сообщений и сформулировать правило, которое его определяет.

Задание 2. Модифицировать программу так, чтобы порядок вывода сообщений соответствовал номерам процессов. Сравнить результаты работы двух вариантов программы.

4 Описание алгоритмов и состояние реализации

Реализован вариант задания 1 на языке C с MPI. Все p процессов вызывают MPI_Init, определяют ранг и число процессов. Каждый процесс с рангом $i > 0$ отправляет одно целое число i процессу 0 с тегом 0. Процесс 0 печатает собственный ранг и выполняет ровно $p - 1$ операций MPI_Recv с источником MPI_ANY_SOURCE, печатая полученное значение и фактического отправителя из MPI_Status. Все процессы завершают работу через MPI_Finalize.

При проверке исправлена граница цикла: ранее выполнялось p приёмов, хотя отправлялось только $p - 1$ сообщений. Лишний приём приводил к зависанию. При $p = 1$ исправленная программа только печатает ранг 0 и завершается.

Для задания 2 создана отдельная программа. В цикле по $i = 1, \dots, p - 1$ источником приёма указывается процесс i вместо MPI_ANY_SOURCE. Все отправители работают параллельно. Процесс 0 по очереди ожидает сообщения от процессов 1, 2, …, $p - 1$ и тем самым задаёт порядок печати.

5 Формальное описание с помощью сетей Петри

Используется параметризованная сеть Петри с цветными фишками. Формализация выполнена с опорой на основные положения аппарата сетей Петри [1]. Для каждого $i \in \{1, \dots, p-1\}$ место A_i содержит одну начальную фишку готовности отправителя, B_i обозначает завершение его обмена. Место R содержит фишку состояния принимающего процесса (k, L): число принятых сообщений и последовательность уже напечатанных рангов. Начальное место Z содержит одну фишку готовности процесса 0. Место F обозначает завершение приёма.

Переход $t_0 : Z \to R$ печатает 0 и создаёт фишку $(0, [0])$. Для каждого $i > 0$ переход обмена имеет входы A_i, R и выходы B_i, R:

$$A_i + R(k, L) \xrightarrow{t_i} B_i + R(k+1, L \parallel [i]). \tag{1}$$

Выражение (1) задаёт функционал t_i: передать ранг i, принять его процессом 0 и напечатать. Обмен отправителя с получателем здесь моделируется одним совместным переходом; внутренняя буферизация MPI абстрагирована. В неупорядоченном варианте допустим любой ещё не сработавший t_i. В упорядоченном варианте дополнительное условие перехода: $i = k + 1$. Переход $t_f : R \to F$ разрешён только при $k = p - 1$; он завершает фазу приёма.

Каждый t_i срабатывает ровно один раз, поскольку фишка A_i имеется в одном экземпляре. Фишка R задаёт последовательную печать процессом 0. Итоговый список содержит все ранги ровно по одному разу. При $p = 1$ после t_0 сразу разрешён t_f.

Место для графической схемы описанной сети Петри: места Z, R, F, A_i, B_i и переходы t_0, t_i, t_f.

6 Листинги программ

Полный листинг программы с неупорядоченным приёмом приведён в приложении А, листинг программы с упорядоченным приёмом --- в приложении Б. Вынесение листингов в приложения позволяет не перегружать основную часть отчёта.

7 Проверка и примеры работы

7.1. Ход выполнения задания 1

Программа собрана средствами CMake. Цель task1_unordered компилируется как программа на языке C11 и связывается с библиотекой MPI. Для компиляторов GNU и Clang включены предупреждения -Wall, -Wextra и -Wpedantic. Сборка завершилась без ошибок.

После инициализации MPI каждый процесс определяет свой ранг. Процессы с рангами больше нуля передают свои номера процессу 0. Нулевой процесс сначала выводит 0, затем выполняет $p - 1$ операций MPI_Recv. В каждой операции источник задан константой MPI_ANY_SOURCE, поэтому может быть принято сообщение от любого ещё не обслуженного отправителя.

Для начальной проверки программа запущена на одном и двух процессах. При $p = 1$ отправители отсутствуют, поэтому выводится только 0. При $p = 2$ процесс 0 получает единственное сообщение от процесса 1. Результат сборки и запусков представлен на рисунке 1.

```
❯ cmake --build build
  mpirun -np 1 ./build/task1_unordered
  mpirun -np 2 ./build/task1_unordered
[100%] Built target task1_unordered
rank: 0
rank: 0
Received rank 1, Sender 1
```

Рисунок 1 – Сборка и запуск неупорядоченной программы на одном и двух процессах

Затем выполнено пять запусков при $p = 8$. В каждом опыте программа завершилась, нулевой ранг был напечатан первым, а каждый из рангов 1--7 встретился ровно один раз. Полученные последовательности приведены в

таблице 1.

Таблица 1 – Последовательности рангов в пяти запусках на восьми процессах

Запуск	Ранги в порядке печати
1	0, 7, 3, 5, 2, 6, 4, 1
2	0, 2, 6, 7, 3, 4, 1, 5
3	0, 1, 2, 3, 4, 5, 6, 7
4	0, 1, 4, 2, 7, 3, 6, 5
5	0, 7, 1, 3, 4, 5, 6, 2

Первым всегда печатается 0. Далее вывод соответствует порядку, в котором операции приёма выбирают подходящие сообщения. Источник MPI_ANY_SOURCE допускает приём от любого процесса, поэтому порядок отправителей не задаётся. На результат влияют планирование процессов и работа MPI; нельзя гарантировать ни возрастание рангов, ни порядок начала отправок, ни строгую сортировку по времени физического прихода сообщений. Одинаковые последовательности при повторных запусках тоже допустимы. Передаются сообщения с целыми числами, а не сигналы ОС. Полный вывод пяти запусков показан на рисунке 2.

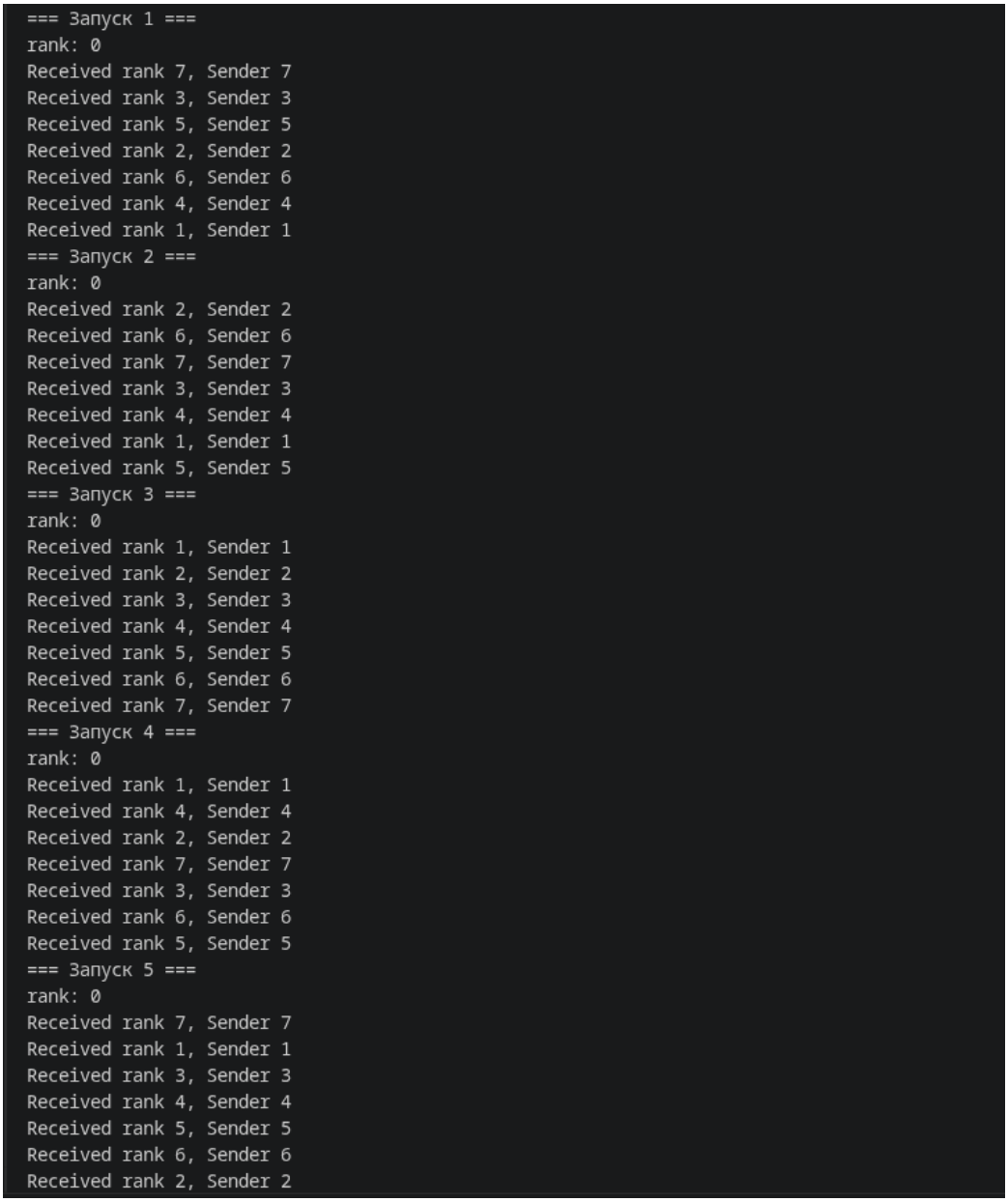


Рисунок 2 – Пять запусков неупорядоченной программы на восьми процессах

7.2. Ход выполнения задания 2

Вторая программа получена изменением источника в операции MPI_Recv: вместо произвольного отправителя MPI_ANY_SOURCE используется значение счётчика цикла i. Алгоритм отправителей и количество операций приёма остались без изменений. Таким образом, на первой итерации процесс 0 ожидает сообщение от процесса 1, на второй --- от процесса 2 и далее до процесса $p - 1$. Сообщение от процесса с большим рангом может

быть готово раньше, но будет обработано только после всех отправителей с меньшими рангами.

Программа собрана средствами CMake и сначала проверена при $p = 1, 2, 4$. При одном процессе выводится только 0; при двух и четырёх процессах получены строго возрастающие последовательности рангов (рисунок 3).

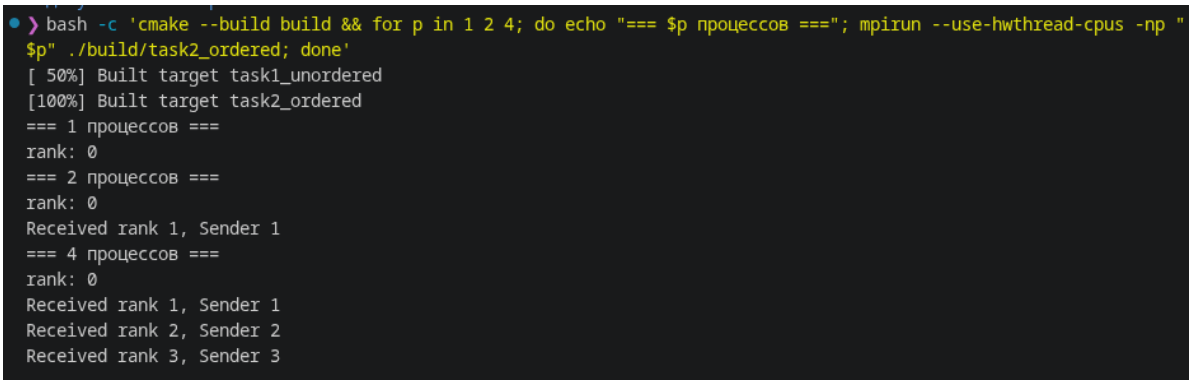


Рисунок 3 – Сборка и запуск упорядоченной программы на одном, двух и четырёх процессах

Для проверки устойчивости результата выполнены три запуска при $p = 8$. Во всех запусках получена одна последовательность $0, 1, 2, 3, 4, 5, 6, 7$, что подтверждает заданный программой порядок приёма. Результаты повторных запусков представлены на рисунке 4.

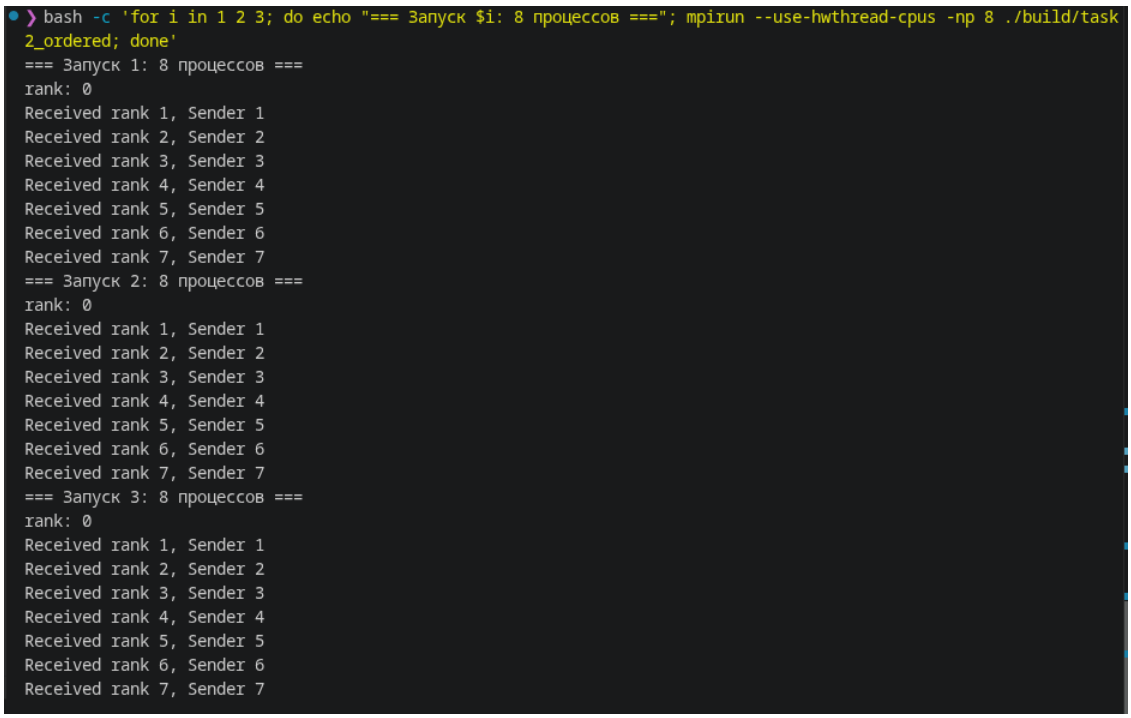


Рисунок 4 – Три запуска упорядоченной программы на восьми процессах

Дополнительно выполнен функциональный запуск при $p = 16$. Поскольку используемый процессор предоставляет 12 аппаратных потоков, применён параметр --oversubscribe. Этот опыт подтверждает корректность порядка $0, 1, \ldots, 15$, но из-за переподписки не используется для оценки производительности. Результат показан на рисунке 5.

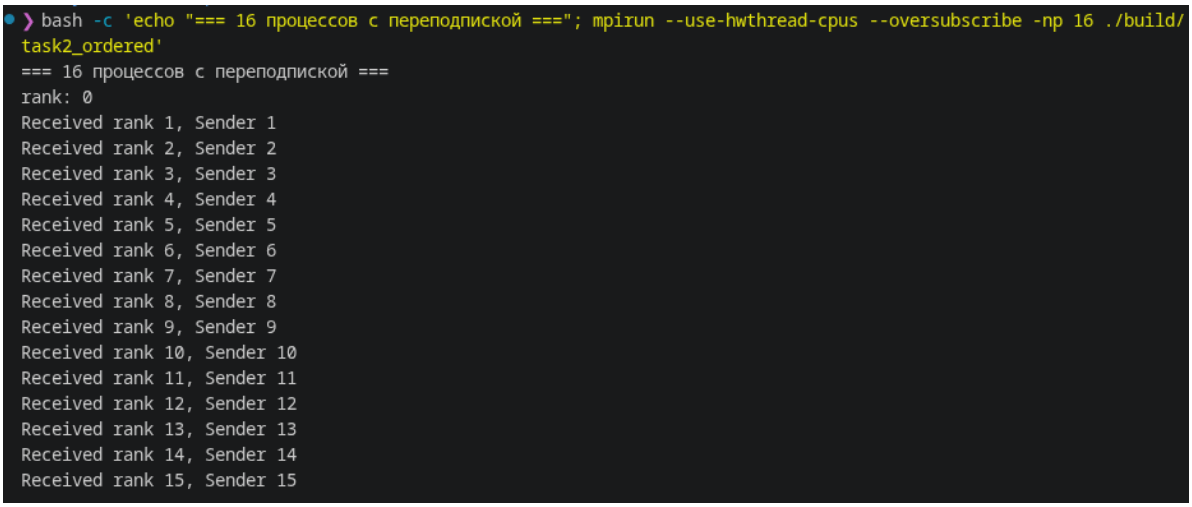


Рисунок 5 – Запуск упорядоченной программы на шестнадцати процессах с переподпиской

8 Методика измерения времени

Замеры времени пока не выполнены: текущий код не содержит измерений и параметра объёма данных. Для выполнения общих требований к отчёту нужна отдельная экспериментальная модификация обеих программ. Обозначим через n число целых чисел в сообщении каждого отправителя; например, $n = 1, 1024, 65536$. В исходной постановке $n = 1$. Подготовку массивов и печать следует вынести за измеряемый участок. Перед обменом выполнить MPI_Barrier, затем на каждом процессе снять MPI_Wtime, выполнить обмен и повторно снять время. За время опыта принять максимум локальных длительностей, вычисленный через MPI_Reduce с операцией MPI_MAX после замера. Полученные ранги сохранять и печатать после окончания измерений.

Для каждой пары (p, n) провести прогрев и, например, десять измерений; в таблицу внести медиану. Проверить $p = 1, 2, 4, 8, 16$ и при необходимости 32. Зафиксировать модель CPU, число физических ядер, ОС, версию MPI, параметры сборки и размещение процессов. Запуски с числом процессов выше числа доступных ядер отмечать отдельно: переподписка влияет на время.

При фиксированном n суммарная передача равна $(p - 1)n$ целых чисел: при изменении p меняется сама работа. При $p = 1$ обмен отсутствует. Поэтому отношение $T(1, n)/T(p, n)$ для исходного задания нельзя интерпретировать как ускорение решения одной и той же задачи. Для содержательного исследования ускорения требуется отдельно определить фиксированный общий объём N, распределяемый по отправителям, и последовательный алгоритм обработки того же объёма с временем $T_{\text{seq}}(N)$. Такой эксперимент является расширением задания; его постановку следует согласовать с преподавателем. Ускорение и эффективность вычисляются по формулам (2):

$$S(p, N) = \frac{T_{\text{seq}}(N)}{T_p(N)}, \qquad E(p, N) = \frac{S(p, N)}{p}. \tag{2}$$

Для исходного обмена полезно сравнивать время двух вариантов при одинаковых (p, n), не обещая наличия ускорения.

9 Таблицы результатов

Таблицу 2 необходимо заполнить отдельно для каждого выбранного n. Прочерки обозначают отсутствие измерений, а не нулевое время.

Таблица 2 – Шаблон результатов измерения времени

p	$T_{\text{unordered}}$, с	T_{ordered}, с
1	---	---
2	---	---
4	---	---
8	---	---
16	---	---
32 (при необходимости)	---	---

10 Графики времени и ускорения

График $T(p, n)$: число процессов по оси X, время по оси Y; отдельные кривые для объёмов данных и двух алгоритмов.

График времени от объёма данных при фиксированных p.

После определения сопоставимой последовательной базы: графики $S(p, N)$ и при необходимости $E(p, N)$. Указать определение нагрузки и базового времени.

11 Сравнение и выводы

Проверенный вариант задания 1 обеспечивает передачу рангов всех ненулевых процессов процессу 0 и завершается корректно после исправления числа приёмов. Повторные запуски подтвердили изменение порядка вывода. В задании 2 последовательность $0, 1, \ldots, p - 1$ обеспечивается явным выбором отправителя. Запуски на 1, 2, 4, 8 и 16 процессах подтвердили возрастающий порядок вывода; три повторных опыта при восьми процессах дали одинаковый результат.

Приём от заданного источника может ожидать конкретный процесс, даже если другие сообщения уже доступны. В обоих алгоритмах все сообщения принимает один процесс, что ограничивает масштабируемость. Какой вариант быстрее на данной машине, можно установить только измерениями. После заполнения таблиц и графиков необходимо описать наблюдаемые изменения времени, возможные точки перелома, влияние размера сообщения и числа ядер. Выводы об ускорении до проведения этих экспериментов не формулируются.

12 Список использованных источников

1 Мараховский В. Б., Розенблюм Л. Я., Яковлев А. В. Моделирование параллельных процессов. Сети Петри. СПб.: Профессиональная литература, 2014.

ПРИЛОЖЕНИЕ А

Листинг программы с неупорядоченным приёмом

```
// Автор: Марк Пархоменко
// Дата: 27.10.2026

#include <stdio.h>
#include <mpi.h>

int main(int argc, char **argv)
{
   int rank;
   int process_count;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &process_count);

   if (rank == 0)
   {
      printf("rank: %d\n", rank);

      for (int i = 1; i < process_count; i++)
      {

         int received_rank;
         MPI_Status status;
```

```
        MPI_Recv(
            &received_rank,
            1,
            MPI_INT,
            MPI_ANY_SOURCE,
            0,
            MPI_COMM_WORLD,
            &status
        );

        printf("Received rank %d, Sender %d\n",
            received_rank,
            status.MPI_SOURCE);
    }
}

else
{
    MPI_Send(
        &rank,
        1,
        MPI_INT,
        0,
        0,
        MPI_COMM_WORLD
    );
}
```

```
    MPI_Finalize();

    return 0;
}
```

ПРИЛОЖЕНИЕ Б

Листинг программы с упорядоченным приёмом

```
// Автор: Марк Пархоменко
// Дата: 27.10.2026

#include <stdio.h>
#include <mpi.h>

int main(int argc, char **argv)
{
   int rank;
   int process_count;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &process_count);

   if (rank == 0)
   {
      printf("rank: %d\n", rank);

      for (int i = 1; i < process_count; i++)
      {

         int received_rank;
         MPI_Status status;
```

```
        MPI_Recv(
            &received_rank,
            1,
            MPI_INT,
            i,
            0,
            MPI_COMM_WORLD,
            &status
        );

        printf("Received rank %d, Sender %d\n",
            received_rank,
            status.MPI_SOURCE);
    }
}

else
{
    MPI_Send(
        &rank,
        1,
        MPI_INT,
        0,
        0,
        MPI_COMM_WORLD
    );
}
```

```
    MPI_Finalize();

    return 0;
}
```